

MODUL PRAKTIKUM

Prediksi Awal Kesehatan Daun Berbasis Citra Digital

**PROGRAM STUDI
INSTITUSI PENDIDIKAN
2024**

Kata Pengantar

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulisan "Modul Praktikum Prediksi Awal Kesehatan Daun Berbasis Citra Digital" ini dapat diselesaikan dengan baik.

Modul ini dirancang secara khusus untuk memberikan pengalaman praktis secara komprehensif dari awal hingga akhir (end-to-end) dalam membangun sebuah sistem Machine Learning berbasis Computer Vision. Melalui studi kasus klasifikasi kondisi daun menjadi 5 (lima) kelas, pembaca akan dipandu mulai dari tahapan pemuatan data, eksplorasi data, perancangan model jaringan saraf tiruan (ResNet9), pelatihan model, hingga tahapan *deployment* menjadi sebuah aplikasi web sederhana menggunakan Streamlit.

Penggunaan Google Colab sebagai environment utama dalam praktikum ini diharapkan dapat memudahkan pembaca dalam mempraktekkan materi tanpa perlu memikirkan spesifikasi perangkat keras yang tinggi, karena seluruh komputasi dilakukan melalui teknologi *cloud*.

Kami menyadari bahwa modul ini tidak luput dari kekurangan. Oleh karena itu, segala bentuk kritik dan saran yang membangun sangat kami harapkan guna penyempurnaan di masa yang akan datang. Perlu ditegaskan pula bahwa model yang dibangun dalam modul ini berfungsi sebagai purwarupa (prototype) untuk *screening* awal, dan bukan sebagai diagnosis agrikultural yang final.

Akhir kata, semoga modul ini dapat memberikan manfaat yang sebesar-besarnya bagi para pembaca, khususnya bagi pihak yang sedang mendalami implementasi Machine Learning dan Computer Vision di dunia nyata.

Penyusun

Daftar Isi

Kata Pengantar	i
Daftar Gambar	iii
Daftar Tabel	iv
Daftar Potongan Kode	v
Daftar Output Kode	vi
1 Pendahuluan	1
1.1 Gambaran Umum	1
1.2 Tujuan Praktikum	2
1.3 Batasan Sistem	2
1.4 Persiapan Lingkungan Praktikum	2
2 Persiapan dan Eksplorasi Data	4
2.1 Data Loading dengan KaggleHub	4
2.1.1 Pengambilan Data	4
2.1.2 Mengecek Struktur Folder Dataset	5
2.2 Persiapan Environment	6
2.2.1 Mengimpor Library	6
2.2.2 Menginstal Library Tambahan	7
2.3 Exploratory Data Analysis (EDA)	8
2.3.1 Menentukan Lokasi Folder Dataset	8
2.3.2 Menghitung Total Gambar dalam Dataset	9
2.3.3 Menampilkan Contoh Gambar Dataset	10
2.3.4 Identifikasi Kelas Dataset dan Memisahkan Kelas Penyakit dan Kelas Sehat	11
2.3.5 Identifikasi Data yang Akan Digunakan	14
2.3.6 Visualisasi Distribusi Data yang Digunakan	17
2.3.7 Visualisasi Total Gambar per Kelas	18
2.3.8 Ringkasan Jumlah Data Training dan Validation	19

Daftar Gambar

1.1	Gambaran Alur Kerja (End-to-End Workflow)	1
1.2	Detail Pipeline Pemrosesan Data dan Pemodelan	1
2.1	Contoh gambar daun dari dataset	11
2.2	Visualisasi Distribusi Jumlah Gambar pada 5 Kelas Target	18
2.3	Visualisasi Total Gambar pada Setiap Kelas Target	19

Daftar Tabel

2.1	Tabel Daftar Tanaman	13
2.2	Tabel Kelas Sehat	13
2.3	Tabel Kelas Penyakit	14
2.4	Tabel Rincian Data Target (Daun Tomat)	17

Daftar Potongan Kode

2.1	Mengunduh dataset dari KaggleHub	4
2.2	Mengecek struktur folder dataset	5
2.3	Mengimpor library yang dibutuhkan	6
2.4	Menginstal library torchsummary	7
2.5	Menentukan lokasi folder dataset	8
2.6	Menghitung total gambar dalam dataset	9
2.7	Menampilkan contoh gambar dataset	10
2.8	Identifikasi dan filter kelas dataset	11
2.9	Menentukan dan menghitung data target	14
2.10	Membuat visualisasi distribusi data target	17
2.11	Membuat visualisasi total gambar	18
2.12	Menghitung ringkasan akhir jumlah data	19

Daftar Output Kode

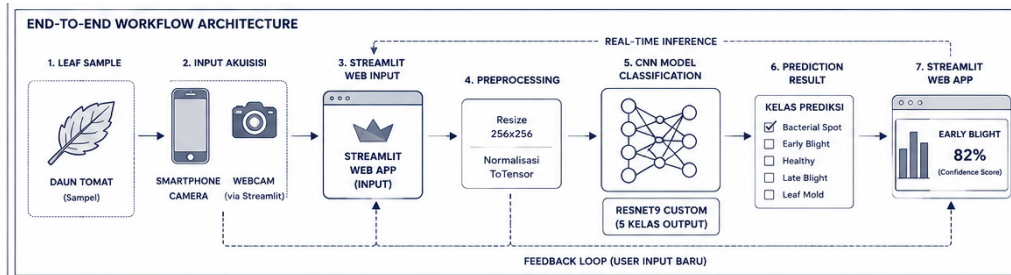
1	Output pengunduhan dataset	4
2	Output struktur folder dataset	5
3	Output impor library	6
4	Output instalasi torchsummary	7
5	Output lokasi folder dataset	9
6	Output total gambar dalam dataset	9
7	Output ringkasan jumlah kelas	13
8	Output ringkasan data target	16
9	Output ringkasan akhir data	20

Bab 1

Pendahuluan

1.1 Gambaran Umum

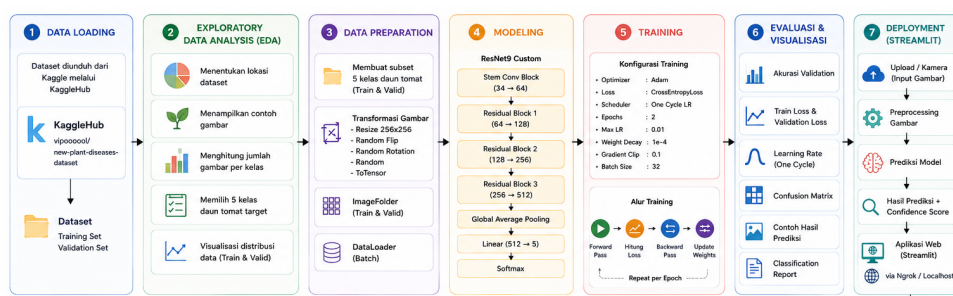
Pada praktikum ini, kita akan membangun sebuah purwarupa (prototype) sistem Machine Learning berbasis Computer Vision yang bertujuan untuk memprediksi secara dini kondisi kesehatan daun tomat. Praktikum ini menggunakan pendekatan end-to-end, mulai dari pemuatan data (*data loading*) hingga fase *deployment* dalam bentuk aplikasi web interaktif.



Gambar 1.1: Gambaran Alur Kerja (End-to-End Workflow)

Sebagaimana diilustrasikan pada Gambar 1.1, alur kerja dalam proyek Machine Learning ini merupakan proses berkesinambungan yang berawal dari pengumpulan dan penyiapan dataset daun tomat, diikuti proses perancangan serta pelatihan model jaringan saraf tiruan. Setelah model dievaluasi dan dianggap memadai, sistem diakhiri dengan proses integrasi (deployment) ke dalam platform aplikasi web interaktif agar model siap digunakan oleh para pengguna akhir.

Sistem ini dirancang tidak untuk memberikan diagnosis final layaknya pandangan pakar agrikultur, melainkan digunakan sebagai alat bantu *screening* awal berdasarkan ciri-ciri visual yang tampak pada gambar daun.



Gambar 1.2: Detail Pipeline Pemrosesan Data dan Pemodelan

Lebih rinci lagi, struktur (pipeline) komputasi di bawah kap mesin dapat divisualisasikan pada Gambar 1.2. Alur komputasi ini memperlihatkan transformasi gambar asli ke dalam matriks angka (Tensor) yang sesuai dengan kerangka kerja PyTorch, yang selanjutnya akan diproses secara matematis oleh lapisan-lapisan (layers) pada model arsitektur ResNet9 guna membedakan pola penyakit pada daun tomat.

1.2 Tujuan Praktikum

Adapun tujuan utama dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memberikan pengalaman praktik alur kerja (workflow) pembuatan model Machine Learning secara utuh.
2. Membuat model Machine Learning untuk mengklasifikasikan gambar daun tomat.
3. Menampilkan hasil prediksi model dalam bentuk aplikasi web sederhana.
4. Memahami bahwa model memiliki batasan dan hanya valid untuk data yang sesuai dengan domain pelatihan.

1.3 Batasan Sistem

Untuk memastikan fokus pembelajaran tetap terarah, sistem yang akan dibangun memiliki beberapa batasan:

- Dataset difokuskan secara eksklusif pada **5 kelas kondisi daun tomat**:
 1. Bacterial Spot
 2. Early Blight
 3. Late Blight
 4. Leaf Mold
 5. Healthy (Sehat)
- Sistem tidak dapat digunakan untuk memprediksi jenis tanaman lain.
- Model yang dilatih belum dilengkapi dengan kelas penolakan (seperti kelas 'Unknown' atau 'Non Leaf'), sehingga input gambar diasumsikan selalu berupa daun tomat.

1.4 Persiapan Lingkungan Praktikum

Pada praktikum ini, spesifikasi komputer lokal yang tinggi tidak diwajibkan karena seluruh proses komputasi, termasuk pelatihan model yang membutuhkan GPU, akan dilakukan di *cloud*. Berikut adalah *tools* dan lingkungan yang perlu dipersiapkan:

Berikut adalah rincian masing-masing teknologi yang menjadi tulang punggung praktikum ini:

- **Google Colab**: Lingkungan Jupyter Notebook gratis berbasis cloud dari Google yang menyediakan akses ke GPU. Digunakan untuk mengeksekusi kode Python, melatih model, dan menjalankan **server** aplikasi.
- **KaggleHub**: Library untuk mengunduh dataset secara langsung dari Kaggle ke environment Google Colab.

- **PyTorch dan Torchvision:** Framework Deep Learning yang akan digunakan untuk mengelola dataset dan membangun arsitektur jaringan saraf tiruan (ResNet9).
- **Streamlit:** Framework open-source berbasis Python untuk membuat antarmuka web interaktif khusus Machine Learning.
- **Ngrok:** Layanan *tunneling* yang memungkinkan aplikasi Streamlit yang berjalan di *localhost* Google Colab dapat diakses melalui internet menggunakan tautan publik.

Pastikan Anda telah memiliki akun Google untuk menggunakan Google Colab. Pada bab selanjutnya, kita akan mulai mempersiapkan **environment** praktikum.

Bab 2

Persiapan dan Eksplorasi Data

Bab ini akan memandu Anda dalam mempersiapkan seluruh kebutuhan data yang akan digunakan pada praktikum ini. Terdapat tiga tahapan utama yang akan dilalui, yaitu pengambilan data (*data loading*) dari KaggleHub, persiapan *environment* dengan mengimpor seluruh *library* yang dibutuhkan, serta eksplorasi data awal (*Exploratory Data Analysis* / EDA) untuk memahami karakteristik dataset yang akan digunakan.

2.1 Data Loading dengan KaggleHub

Langkah pertama yang harus dilakukan adalah mengambil dataset gambar daun tanaman dari KaggleHub. Dataset akan digunakan sebagai bahan utama untuk melatih model *Machine Learning* agar mampu mengenali kondisi kesehatan tanaman berdasarkan citra daun.

2.1.1 Pengambilan Data

Tahap awal untuk memulai praktikum ini adalah mendatangkan bahan baku berupa ribuan foto tanaman dari repositori *online* ke dalam lingkungan komputasi kita. Dataset yang digunakan berasal dari Kaggle dengan nama `vipooooool/new-plant-diseases-dataset`.

KaggleHub adalah *library* Python yang memungkinkan kita mengunduh dataset dari Kaggle secara langsung tanpa perlu mengunduh secara manual melalui *browser*. Dengan menggunakan KaggleHub, dataset akan secara otomatis diunduh dan diekstrak ke dalam *path* yang dapat langsung digunakan.

Berikut adalah kode untuk mengunduh dataset:

```
1 import kagglehub
2
3 # Mengunduh dataset Plant Disease dari KaggleHub
4 dataset_path = kagglehub.dataset_download(
5     "vipooooool/new-plant-diseases-dataset"
6 )
7
8 # Menampilkan lokasi dataset yang berhasil diunduh
9 print("Dataset berhasil diunduh.")
10 print("Lokasi dataset:", dataset_path)
```

Kode 2.1: Mengunduh dataset dari KaggleHub

Ketika kode di atas dijalankan, KaggleHub akan mengunduh dataset dan menyimpannya ke dalam direktori *cache* lokal. Variabel `dataset_path` akan menyimpan lokasi lengkap dari dataset yang telah diunduh. Lokasi ini nantinya akan digunakan pada tahap-tahap selanjutnya untuk mengakses gambar-gambar yang akan diproses.

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Dataset berhasil diunduh.
2 Lokasi dataset: /kaggle/input/new-plant-diseases-dataset
```

Output 1: Output pengunduhan dataset

Catatan Penting

Pastikan Anda telah terhubung ke internet saat menjalankan kode di atas. Dataset yang diunduh memiliki ukuran yang cukup besar, sehingga proses pengunduhan mungkin memerlukan waktu beberapa saat tergantung pada kecepatan koneksi internet Anda.

2.1.2 Mengecek Struktur Folder Dataset

Setelah dataset berhasil diunduh, langkah berikutnya adalah mengecek isi folder utama dataset. Tahap ini penting karena setiap dataset Kaggle bisa memiliki struktur folder yang berbeda. Dengan melihat isi folder utama, kita dapat menentukan lokasi folder *train*, *valid*, dan *test* yang akan digunakan pada proses pelatihan model.

Berikut adalah kode untuk mengecek struktur folder dataset:

```
1 import os
2
3 # Menampilkan lokasi dataset
4 print("Lokasi dataset:")
5 print(dataset_path)
6
7 # Mengecek isi folder utama dataset
8 print("\nIsi folder utama dataset:")
9
10 for item in os.listdir(dataset_path):
11     item_path = os.path.join(dataset_path, item)
12
13     if os.path.isdir(item_path):
14         print(f"[Folder] {item}")
15     else:
16         print(f"[File] {item}")
```

Kode 2.2: Mengecek struktur folder dataset

Setelah kode di atas dijalankan, akan terlihat struktur folder dataset sebagai berikut:

```
1 Lokasi dataset:
2 /kaggle/input/new-plant-diseases-dataset
3
4 Isi folder utama dataset:
5 [Folder] New Plant Diseases Dataset(Augmented)
6 [Folder] new plant diseases dataset(augmented)
7 [Folder] test
```

Output 2: Output struktur folder dataset

Dari output tersebut, kita dapat melihat bahwa dataset terdiri dari tiga folder utama. Dua di antaranya merupakan folder yang berisi data *training* dan *validation* yang sudah di-*augmentasi*, serta satu folder *test* yang berisi data pengujian. Pada praktikum ini, kita akan menggunakan folder yang berisi data *augmented* untuk melatih dan mengevaluasi model.

2.2 Persiapan Environment

Sebelum memulai proses pengolahan data dan pemodelan, kita perlu menyiapkan *environment* dengan mengimpor seluruh *library* yang dibutuhkan. *Library* yang akan digunakan meliputi *library* untuk pengelolaan data, visualisasi, dan *deep learning*.

2.2.1 Mengimpor Library

Berikut adalah kode untuk mengimpor seluruh *library* yang dibutuhkan dalam praktikum ini:

```
1 # Library untuk pengelolaan file dan data
2 import os
3 import numpy as np
4 import pandas as pd
5
6 # Library untuk visualisasi
7 import matplotlib.pyplot as plt
8 from PIL import Image
9
10 # Library utama PyTorch
11 import torch
12 import torch.nn as nn
13 import torch.nn.functional as F
14
15 # Library untuk pengolahan dataset dan DataLoader
16 from torch.utils.data import DataLoader
17 from torchvision.datasets import ImageFolder
18 import torchvision.transforms as transforms
19 from torchvision.utils import make_grid
20
21 # Library untuk melihat ringkasan arsitektur model
22 from torchsummary import summary
23
24 # Menampilkan plot langsung di notebook
25 %matplotlib inline
26
27 print("Semua library berhasil diimport.")
```

Kode 2.3: Mengimpor library yang dibutuhkan

Setelah kode di atas dijalankan, akan muncul output berikut:

```
1 Semua library berhasil diimport.
```

Output 3: Output impor library

Penjelasan masing-masing *library* yang diimpor:

- `os` — Digunakan untuk mengelola file dan direktori.
- `numpy` — Library untuk komputasi numerik dan operasi matriks.
- `pandas` — Library untuk manipulasi dan analisis data tabular.

- `matplotlib.pyplot` — Library untuk membuat grafik dan visualisasi data.
- `PIL.Image` — Library untuk membuka dan memproses gambar.
- `torch` — Library utama PyTorch untuk *deep learning*.
- `torch.nn` — Modul PyTorch untuk membangun arsitektur jaringan saraf tiruan.
- `torch.nn.functional` — Modul PyTorch untuk fungsi-fungsi aktivasi dan *loss*.
- `DataLoader` — Digunakan untuk memuat data dalam bentuk *batch*.
- `ImageFolder` — Dataset loader khusus untuk data gambar terstruktur dalam folder.
- `transforms` — Modul untuk melakukan transformasi gambar.
- `make_grid` — Utility untuk menampilkan kumpulan gambar dalam bentuk grid.
- `torchsummary` — Library untuk menampilkan ringkasan arsitektur model.

2.2.2 Menginstal Library Tambahan

Beberapa *library* mungkin belum tersedia di *environment* Google Colab secara bawaan. Oleh karena itu, kita perlu menginstalnya terlebih dahulu menggunakan `pip`. Salah satu *library* yang perlu diinstal adalah `torchsummary`:

```
1 !pip install torchsummary -q
2
3 print("Library torchsummary berhasil diinstall.")
```

Kode 2.4: Menginstal library `torchsummary`

Output yang akan tampil:

```
1 Library torchsummary berhasil diinstall.
```

Output 4: Output instalasi `torchsummary`

Catatan Penting

Perintah `!pip install` hanya perlu dijalankan satu kali pada awal sesi Google Colab. Jika *runtime* di-reset, Anda perlu menginstal ulang *library* yang tidak termasuk dalam bawaan Colab.

Setelah semua *library* berhasil diimpor dan diinstal, *environment* praktikum siap digunakan untuk tahap selanjutnya, yaitu eksplorasi data awal (*Exploratory Data Analysis*).

2.3 Exploratory Data Analysis (EDA)

Setelah dataset berhasil dimuat dan *environment* siap, tahap berikutnya adalah melakukan *Exploratory Data Analysis* (EDA). EDA digunakan untuk memahami kondisi awal dataset, seperti jumlah total gambar, struktur folder, jumlah kelas, dan sebaran data pada setiap kelas. Tahap ini penting agar kita mengetahui apakah dataset sudah layak digunakan untuk proses *training* model *Machine Learning*.

2.3.1 Menentukan Lokasi Folder Dataset

Sebelum melakukan EDA, kita perlu menentukan lokasi folder dataset yang akan digunakan. Pada dataset ini, data *training* dan *validation* berada di dalam folder New Plant Diseases Dataset(Augmented). Folder train digunakan untuk melatih model, sedangkan folder valid digunakan untuk mengevaluasi performa model selama proses *training*.

Tahap ini penting karena variabel `train_dir`, `valid_dir`, dan `diseases` akan digunakan pada proses EDA dan *preprocessing* berikutnya.

Berikut adalah kode untuk menentukan lokasi folder dataset:

```
1 import os
2
3 # Path ke folder dataset utama
4 dataset_main = os.path.join(
5     dataset_path,
6     "New Plant Diseases Dataset(Augmented)",
7     "New Plant Diseases Dataset(Augmented)"
8 )
9
10 # Path ke folder training dan validation
11 train_dir = os.path.join(dataset_main, "train")
12 valid_dir = os.path.join(dataset_main, "valid")
13
14 # Menampilkan lokasi folder
15 print("Folder dataset utama :", dataset_main)
16 print("Folder training      :", train_dir)
17 print("Folder validation     :", valid_dir)
18
19 # Mendapatkan daftar kelas (nama folder dalam train)
20 diseases = os.listdir(train_dir)
21 print("Jumlah kelas dataset :", len(diseases))
```

Kode 2.5: Menentukan lokasi folder dataset

Output yang akan tampil:

```

1 Folder dataset utama : /kaggle/input/new-plant-diseases-dataset/New
  Plant
2                               Diseases Dataset(Augmented)/New Plant Diseases
3                               Dataset(Augmented)
4 Folder training       : /kaggle/input/new-plant-diseases-dataset/New
  Plant
5                               Diseases Dataset(Augmented)/New Plant Diseases
6                               Dataset(Augmented)/train
7 Folder validation     : /kaggle/input/new-plant-diseases-dataset/New
  Plant
8                               Diseases Dataset(Augmented)/New Plant Diseases
9                               Dataset(Augmented)/valid
10 Jumlah kelas dataset : 38

```

Output 5: Output lokasi folder dataset

Dari output di atas, kita dapat melihat bahwa dataset utama memiliki total 38 kelas penyakit tanaman. Namun, untuk praktikum ini kita hanya akan fokus pada 5 kelas daun tomat.

2.3.2 Menghitung Total Gambar dalam Dataset

Setelah mengetahui struktur folder, langkah selanjutnya adalah menghitung total seluruh gambar yang tersedia untuk memastikan kita memiliki cukup banyak contoh foto untuk dipelajari oleh model.

Berikut adalah kode untuk menghitung total gambar dalam dataset:

```

1 # Ekstensi file gambar yang akan dihitung
2 image_extensions = (".jpg", ".jpeg", ".png")
3
4 # Variabel untuk menyimpan jumlah total gambar
5 total_images = 0
6
7 # Menelusuri seluruh folder dan subfolder dataset
8 for root, dirs, files in os.walk(dataset_path):
9     image_files = [
10         file for file in files
11         if file.lower().endswith(image_extensions)
12     ]
13
14     total_images += len(image_files)
15
16 # Menampilkan hasil
17 print("Total gambar dalam dataset:", total_images)

```

Kode 2.6: Menghitung total gambar dalam dataset

Setelah kode dijalankan, akan tampil output berikut:

```

1 Total gambar dalam dataset: 175767

```

Output 6: Output total gambar dalam dataset

Kode di atas akan menelusuri seluruh folder dan subfolder dalam dataset, kemudian menghitung jumlah file gambar dengan ekstensi .jpg, .jpeg, dan .png. Hasilnya menampilkan total seluruh gambar yang tersedia di dalam dataset, yaitu sebanyak **175.767** gambar.

2.3.3 Menampilkan Contoh Gambar Dataset

Setelah mengetahui jumlah total gambar, langkah selanjutnya adalah melihat contoh gambar dari dataset. Hal ini penting untuk memahami karakteristik visual dari data yang akan diproses, seperti ukuran gambar, kualitas resolusi, dan variasi kondisi daun yang ada.

Berikut adalah kode untuk menampilkan contoh gambar dari dataset:

```
1 # Ekstensi gambar yang akan dicari
2 image_extensions = (".jpg", ".jpeg", ".png")
3
4 # Menyimpan seluruh path gambar
5 image_paths = []
6
7 # Menelusuri seluruh folder dataset
8 for root, dirs, files in os.walk(dataset_path):
9     for file in files:
10         if file.lower().endswith(image_extensions):
11             image_paths.append(os.path.join(root, file))
12
13 # Mengecek apakah ada gambar yang ditemukan
14 if len(image_paths) == 0:
15     print("Tidak ada gambar yang ditemukan. Periksa kembali lokasi dataset.")
16 else:
17     # Mengambil gambar pertama sebagai contoh
18     sample_image_path = image_paths[0]
19
20     # Membuka gambar
21     img = Image.open(sample_image_path).convert("RGB")
22
23     # Menampilkan gambar
24     plt.figure(figsize=(5, 5))
25     plt.imshow(img)
26     plt.axis("off")
27     plt.title("Contoh Gambar Dataset")
28     plt.show()
```

Kode 2.7: Menampilkan contoh gambar dataset

Output yang akan tampil setelah kode dijalankan adalah visualisasi gambar sebagai berikut (lihat Gambar 2.1):



Gambar 2.1: Contoh gambar daun dari dataset

Dari output di atas, kita dapat melihat bahwa setiap gambar dalam dataset memiliki ukuran **256×256** piksel. Gambar contoh yang ditampilkan berasal dari kelas `Tomato___Late_blight` (penyakit hawar daun tomat).

Catatan Penting

Pada lingkungan Google Colab, perintah `plt.show()` akan menampilkan gambar langsung di bawah sel kode. Jika Anda menjalankan kode di *terminal* biasa, pastikan *backend* matplotlib telah dikonfigurasi dengan benar untuk menampilkan grafik.

2.3.4 Identifikasi Kelas Dataset dan Memisahkan Kelas Penyakit dan Kelas Sehat

Selanjutnya, kita akan memetakan variasi data dengan memisahkan seluruh kelas menjadi dua kelompok: tanaman sehat dan berpenyakit. Kelas yang mengandung kata `healthy` akan dikategorikan sebagai tanaman sehat, sedangkan sisanya otomatis masuk ke daftar kelas penyakit. Langkah ini penting untuk mengetahui seberapa kompleks variasi dataset yang kita miliki.

```

1
2 # 1. Memisahkan kelas penyakit dan kelas sehat
3
4 diseases_only = [
5     class_name for class_name in diseases
6     if "healthy" not in class_name.lower()
7 ]
8
9 healthy_classes = [
10    class_name for class_name in diseases
11    if "healthy" in class_name.lower()
12 ]
13
14 # 2. Mengambil daftar jenis tanaman dari nama kelas
15
16 def get_plant_name(class_name):
17     """

```

```

18     Fungsi untuk mengambil nama tanaman dari nama kelas dataset.
19
20     Contoh:
21     Tomato__Early_blight -> Tomato
22     Apple__Black_rot -> Apple
23     """
24     if "__" in class_name:
25         return class_name.split("__")[0]
26     return class_name
27
28 plant_names = sorted(list(set([
29     get_plant_name(class_name) for class_name in diseases
30 ])))
31
32 # 3. Menampilkan ringkasan jumlah kelas
33
34 print("Total semua kelas      :", len(diseases))
35 print("Total kelas penyakit   :", len(diseases_only))
36 print("Total kelas sehat      :", len(healthy_classes))
37 print("Total jenis tanaman     :", len(plant_names))
38
39 # 4. Membuat tabel daftar tanaman
40 import pandas as pd
41
42 df_plants = pd.DataFrame({
43     "No": range(1, len(plant_names) + 1),
44     "Jenis Tanaman": plant_names
45 })
46
47 print("\nTabel Daftar Tanaman:")
48 display(df_plants)
49
50 # 5. Membuat tabel kelas penyakit
51
52 df_only_diseases = pd.DataFrame({
53     "No": range(1, len(diseases_only) + 1),
54     "Kelas Penyakit": diseases_only
55 })
56
57 print("\nTabel Kelas Penyakit:")
58 display(df_only_diseases)
59
60 # 6. Membuat tabel kelas sehat
61
62 df_healthy_classes = pd.DataFrame({
63     "No": range(1, len(healthy_classes) + 1),
64     "Kelas Sehat": healthy_classes
65 })
66
67 print("\nTabel Kelas Sehat:")
68 display(df_healthy_classes)

```

Kode 2.8: Identifikasi dan filter kelas dataset

Output teks yang akan tampil setelah kode dijalankan:

```

1 Total semua kelas      : 38
2 Total kelas penyakit   : 26
3 Total kelas sehat      : 12
4 Total jenis tanaman    : 14

```

Output 7: Output ringkasan jumlah kelas

Selain output teks di atas, kode tersebut juga akan menampilkan tiga buah tabel secara interaktif. Berikut ini adalah tabel-tabel yang dihasilkan:

Tabel 2.1: Tabel Daftar Tanaman

No	Jenis Tanaman
1	Apple
2	Blueberry
3	Cherry_(including_sour)
4	Corn_(maize)
5	Grape
6	Orange
7	Peach
8	Pepper,_bell
9	Potato
10	Raspberry
11	Soybean
12	Squash
13	Strawberry
14	Tomato

Tabel 2.2: Tabel Kelas Sehat

No	Kelas Sehat
1	Apple___healthy
2	Blueberry___healthy
3	Cherry_(including_sour)___healthy
4	Corn_(maize)___healthy
5	Grape___healthy
6	Peach___healthy
7	Pepper,_bell___healthy
8	Potato___healthy
9	Raspberry___healthy
10	Soybean___healthy
11	Strawberry___healthy
12	Tomato___healthy

Tabel 2.3: Tabel Kelas Penyakit

No	Kelas Penyakit	No	Kelas Penyakit
1	Apple___Apple_scab	14	Potato___Early_blight
2	Apple___Black_rot	15	Potato___Late_blight
3	Apple___Cedar_apple_rust	16	Squash___Powdery_mildew
4	Cherry_(including_sour)___Powdery_mildew	17	Strawberry___Leaf_scorch
5	Corn_(maize)___Cercospora_leaf_spot Gray_leaf...	18	Tomato___Bacterial_spot
6	Corn_(maize)___Common_rust_	19	Tomato___Early_blight
7	Corn_(maize)___Northern_Leaf_Blight	20	Tomato___Late_blight
8	Grape___Black_rot	21	Tomato___Leaf_Mold
9	Grape___Esca_(Black_Measles)	22	Tomato___Septoria_leaf_spot
10	Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	23	Tomato___Spider_mites Two-spotted_spider_mite
11	Orange___Haunglongbing_(Citrus_greening)	24	Tomato___Target_Spot
12	Peach___Bacterial_spot	25	Tomato___Tomato_Yellow_Leaf_Curl_Virus
13	Pepper_bell___Bacterial_spot	26	Tomato___Tomato_mosaic_virus

2.3.5 Identifikasi Data yang Akan Digunakan

Melihat luasnya cakupan 14 jenis tanaman dan puluhan penyakit dari tabel di atas, melatih AI untuk mengenali semuanya sekaligus tentu membutuhkan komputasi yang sangat besar. Oleh karena itu, agar *prototype* sistem lebih fokus dan efisien, ruang lingkup data akan dibatasi khusus pada komoditas daun Tomat. Secara spesifik, kita hanya akan mengekstrak lima kondisi target: **Bacterial Spot**, **Early Blight**, **Late Blight**, **Leaf Mold**, dan **Healthy**. Mari kita saring *dataset* ini dan hitung berapa banyak "amunisi" gambar yang siap digunakan untuk melatih model!

```

1
2 # 1. Menentukan kelas yang akan digunakan sesuai proposal
3
4 target_classes = {
5     "Tomato___Bacterial_spot": "Bacterial Spot",
6     "Tomato___Early_blight": "Early Blight",
7     "Tomato___Late_blight": "Late Blight",
8     "Tomato___Leaf_Mold": "Leaf Mold",
9     "Tomato___healthy": "Healthy"
10 }
11
12 target_class_folders = list(target_classes.keys())
13
14 # 2. Fungsi untuk mencari folder train dan valid secara otomatis
15
16 def find_split_dir(dataset_path, split_name, target_class_folders):
17     """

```

```

18     Fungsi untuk mencari folder train atau valid yang berisi kelas
19     target.
20     """
21     for root, dirs, files in os.walk(dataset_path):
22         if os.path.basename(root).lower() == split_name.lower():
23             if any(class_folder in dirs for class_folder in
24                 target_class_folders):
25                 return root
26
27     return None
28
29 train_dir = find_split_dir(dataset_path, "train", target_class_folders)
30 valid_dir = find_split_dir(dataset_path, "valid", target_class_folders)
31
32 print("Folder train:", train_dir)
33 print("Folder valid:", valid_dir)
34
35 if train_dir is None:
36     raise FileNotFoundError("Folder train tidak ditemukan atau tidak
37     berisi kelas target.")
38
39 if valid_dir is None:
40     raise FileNotFoundError("Folder valid tidak ditemukan atau tidak
41     berisi kelas target.")
42
43 # 3. Fungsi untuk menghitung jumlah gambar
44
45 image_extensions = (".jpg", ".jpeg", ".png")
46
47 def count_images(folder_path):
48     """
49     Menghitung jumlah file gambar dalam sebuah folder.
50     """
51     total = 0
52
53     if not os.path.exists(folder_path):
54         return 0
55
56     for root, dirs, files in os.walk(folder_path):
57         total += len([
58             file for file in files
59             if file.lower().endswith(image_extensions)
60         ])
61
62     return total
63
64 # 4. Membuat tabel data yang akan digunakan
65 import pandas as pd
66
67 rows = []
68
69 for original_class, output_label in target_classes.items():
70     train_class_path = os.path.join(train_dir, original_class)
71     valid_class_path = os.path.join(valid_dir, original_class)
72
73     train_count = count_images(train_class_path)

```

```

70     valid_count = count_images(valid_class_path)
71
72     plant_name = original_class.split("___")[0]
73     category = "Sehat" if output_label == "Healthy" else "Penyakit"
74
75     rows.append({
76         "Nama Folder Dataset": original_class,
77         "Jenis Tanaman": plant_name,
78         "Output Model": output_label,
79         "Kategori": category,
80         "Jumlah Train": train_count,
81         "Jumlah Valid": valid_count,
82         "Total Gambar": train_count + valid_count
83     })
84
85 df_target_data = pd.DataFrame(rows)
86
87 # 5. Menampilkan ringkasan data yang digunakan
88
89 print("Total kelas yang digunakan :", len(df_target_data))
90 print("Total gambar train           :", df_target_data["Jumlah Train"].
91       sum())
91 print("Total gambar valid           :", df_target_data["Jumlah Valid"].
92       sum())
92 print("Total seluruh gambar        :", df_target_data["Total Gambar"].
93       sum())
93
94 display(df_target_data)

```

Kode 2.9: Menentukan dan menghitung data target

Output teks yang akan tampil setelah kode dijalankan:

```

1 Folder train: /kaggle/input/new-plant-diseases-dataset/New Plant
  Diseases Dataset (Augmented)/New Plant Diseases Dataset (Augmented)/
  train
2 Folder valid: /kaggle/input/new-plant-diseases-dataset/New Plant
  Diseases Dataset (Augmented)/New Plant Diseases Dataset (Augmented)/
  valid
3 Total kelas yang digunakan : 5
4 Total gambar train         : 9281
5 Total gambar valid         : 2319
6 Total seluruh gambar       : 11600

```

Output 8: Output ringkasan data target

Selain output teks di atas, kode tersebut juga akan menghasilkan tabel interaktif yang berisi rincian jumlah gambar untuk masing-masing kelas target. Berikut adalah representasi tabel tersebut:

Tabel 2.4: Tabel Rincian Data Target (Daun Tomat)

Nama Folder Dataset	Jenis Tanaman	Output Model	Kategori	Jml Train	Jml Valid	Total
Tomato__Bacterial_spot	Tomato	Bacterial Spot	Penyakit	1702	425	2127
Tomato__Early_blight	Tomato	Early Blight	Penyakit	1920	480	2400
Tomato__Late_blight	Tomato	Late Blight	Penyakit	1851	463	2314
Tomato__Leaf_Mold	Tomato	Leaf Mold	Penyakit	1882	470	2352
Tomato__healthy	Tomato	Healthy	Sehat	1926	481	2407

2.3.6 Visualisasi Distribusi Data yang Digunakan

Setelah lima kelas target ditentukan, langkah selanjutnya adalah menampilkan distribusi jumlah gambar pada masing-masing kelas. Visualisasi ini berguna untuk melihat perbandingan proporsi data *training* dan *validation* pada kelas **Bacterial Spot**, **Early Blight**, **Late Blight**, **Leaf Mold**, dan **Healthy**. Pemeriksaan ini penting karena ketidakseimbangan jumlah data antar kelas dapat memengaruhi performa model saat proses pelatihan.

```

1
2 # 1. Validasi dataframe
3
4 required_columns = [
5     "Output Model",
6     "Jumlah Train",
7     "Jumlah Valid",
8     "Total Gambar"
9 ]
10
11 for column in required_columns:
12     if column not in df_target_data.columns:
13         raise ValueError(f"Kolom '{column}' tidak ditemukan pada
14                             df_target_data.")
15
16 # 2. Membuat dataframe untuk visualisasi
17
18 plot_data = df_target_data[
19     ["Output Model", "Jumlah Train", "Jumlah Valid"]
20 ].set_index("Output Model")
21
22 # 3. Membuat bar chart jumlah train dan valid
23
24 plt.figure(figsize=(12, 6))
25
26 ax = plot_data.plot(
27     kind="bar",
28     figsize=(12, 6)
29 )
30
31 plt.title("Distribusi Jumlah Gambar pada Data yang Digunakan")
32 plt.xlabel("Kelas Kondisi Daun Tomat")
33 plt.ylabel("Jumlah Gambar")
34 plt.xticks(rotation=30, ha="right")
35 plt.grid(axis="y", linestyle="--", alpha=0.7)
36 plt.legend(title="Jenis Data")

```

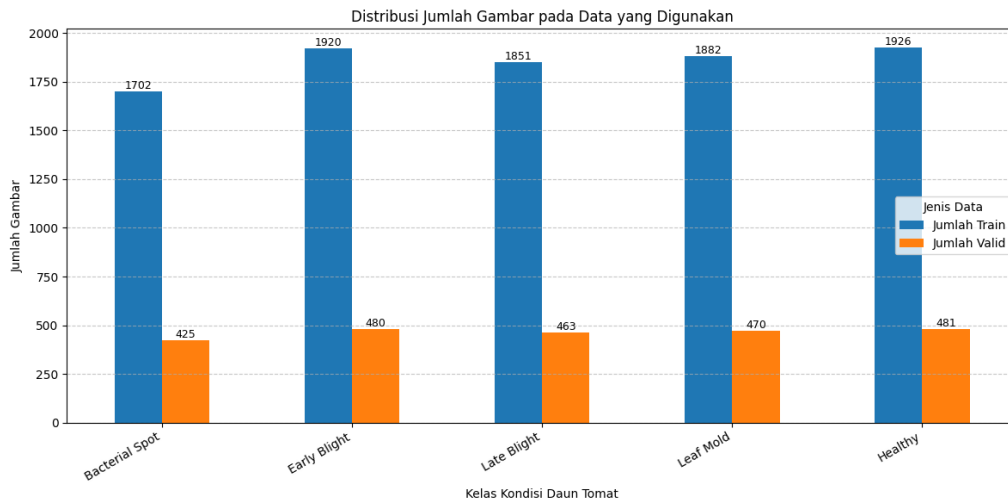
```

36
37 # Menampilkan angka di atas setiap bar
38 for container in ax.containers:
39     ax.bar_label(container, fontsize=9)
40
41 plt.tight_layout()
42 plt.show()

```

Kode 2.10: Membuat visualisasi distribusi data target

Output grafik dari kode di atas adalah sebagai berikut:



Gambar 2.2: Visualisasi Distribusi Jumlah Gambar pada 5 Kelas Target

Dari grafik di atas, terlihat bahwa kelima kelas target memiliki distribusi jumlah gambar yang relatif seimbang, yakni berkisar antara 1.700 hingga 1.900 gambar untuk data *training*, dan 420 hingga 480 gambar untuk data *validation*. Keseimbangan proporsi ini merupakan indikator yang sangat baik. Dengan jumlah sampel yang setara, model AI dapat mempelajari karakteristik setiap kelas secara merata dan mencegah terjadinya bias prediksi ke salah satu kondisi daun tertentu.

2.3.7 Visualisasi Total Gambar per Kelas

Selain membandingkan porsi data *training* dan *validation*, kita juga perlu melihat akumulasi total gambar secara keseluruhan pada setiap kelas. Langkah ini merupakan penegasan akhir untuk memastikan bahwa tidak ada kelas yang mendominasi ataupun kekurangan sampel sebelum kita melangkah ke tahapan prapemrosesan dan pelatihan model.

```

1
2 plt.figure(figsize=(10, 5))
3
4 ax = df_target_data.plot(
5     x="Output Model",
6     y="Total Gambar",
7     kind="bar",
8     legend=False,

```

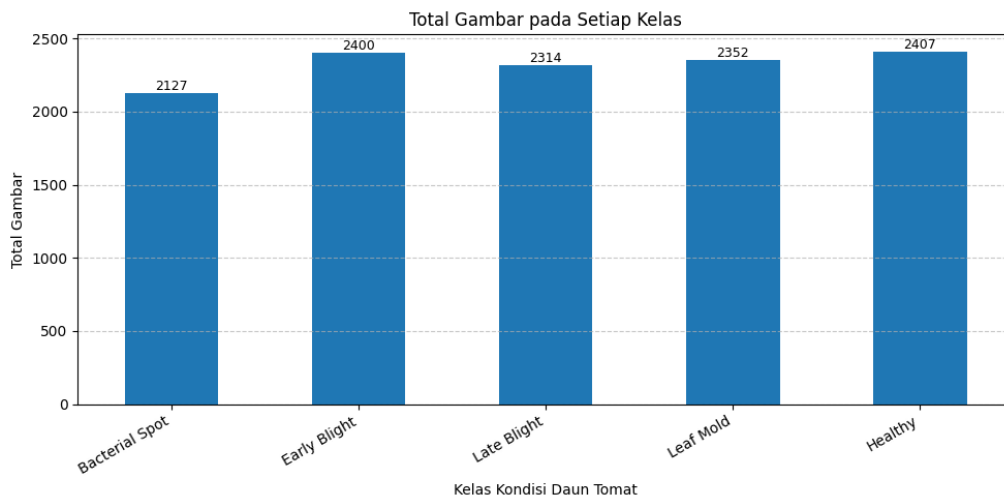
```

9     figsize=(10, 5)
10 )
11
12 plt.title("Total Gambar pada Setiap Kelas")
13 plt.xlabel("Kelas Kondisi Daun Tomat")
14 plt.ylabel("Total Gambar")
15 plt.xticks(rotation=30, ha="right")
16 plt.grid(axis="y", linestyle="--", alpha=0.7)
17
18 # Menampilkan angka di atas bar
19 for container in ax.containers:
20     ax.bar_label(container, fontsize=9)
21
22 plt.tight_layout()
23 plt.show()

```

Kode 2.11: Membuat visualisasi total gambar

Output grafik total gambar adalah sebagai berikut:



Gambar 2.3: Visualisasi Total Gambar pada Setiap Kelas Target

Berdasarkan grafik di atas, kelima kelas target secara konsisten memiliki akumulasi total antara 2.100 hingga 2.400 gambar. Angka yang proporsional ini memastikan bahwa *dataset* kita sudah sepenuhnya ideal dan siap untuk diproses lebih lanjut ke tahap pemodelan.

2.3.8 Ringkasan Jumlah Data Training dan Validation

Sebagai penutup dari tahapan eksplorasi data, kita akan merangkum kembali angka final keseluruhan gambar yang telah disaring. Ringkasan ini memastikan kepastian total data *training* dan *validation* yang akan dilibatkan secara langsung ke dalam proses pelatihan dan evaluasi model.

```

1
2 # Menghitung total gambar pada data training
3 n_train = df_target_data["Jumlah Train"].sum()
4

```

```

5 # Menghitung total gambar pada data validation
6 n_valid = df_target_data["Jumlah Valid"].sum()
7
8 # Menghitung total seluruh gambar yang digunakan
9 n_total = df_target_data["Total Gambar"].sum()
10
11 # Menampilkan ringkasan jumlah data
12 print("Jumlah gambar untuk training   :", n_train)
13 print("Jumlah gambar untuk validation :", n_valid)
14 print("Total gambar yang digunakan    :", n_total)

```

Kode 2.12: Menghitung ringkasan akhir jumlah data

Output dari perhitungan di atas adalah sebagai berikut:

```

1 Jumlah gambar untuk training   : 9281
2 Jumlah gambar untuk validation : 2319
3 Total gambar yang digunakan    : 11600

```

Output 9: Output ringkasan akhir data